

Binary Exponentiation by Factoring

Consider a problem of computing $a^x \pmod{2^d}$, given integers a , x , y and $d \geq 3$, where x is odd.

The algorithm below allows to solve this problem with $O(d)$ additions and binary operations and a single multiplication by y .

Due to the structure of the multiplicative group modulo 2^d , any number x such that $x \equiv 1 \pmod{4}$ can be represented as

$$x \equiv b^{L(x)} \pmod{2^d},$$

where $b \equiv 5 \pmod{8}$. Without loss of generality we assume that $x \equiv 1 \pmod{4}$, as we can reduce $x \equiv 3 \pmod{4}$ to $x \equiv 1 \pmod{4}$ by substituting $x \mapsto -x$ and $a \mapsto (-1)^y a$. In this notion, a^x is represented as

$$a^x \equiv a^{b^{L(x)}} \pmod{2^d}.$$

The core idea of the algorithm is to simplify the computation of $L(x)$ and $b^{L(x)}$ using the fact that we're working modulo 2^d . For reasons that will be apparent later on, we'll be working with $4L(x)$ rather than $L(x)$, but taken modulo 2^d instead of 2^{d-2} .

In this article, we will cover the implementation for 32-bit integers. Let

- `mbin_log_32(r, x)` be a function that computes $r+4L(x) \pmod{2^d}$;
- `mbin_exp_32(r, x)` be a function that computes $r b^{\frac{x}{4}} \pmod{2^d}$;
- `mbin_power_odd_32(a, x, y)` be a function that computes $a^x \pmod{2^d}$.

Then `mbin_power_odd_32` is implemented as follows:

```
uint32_t mbin_power_odd_32(uint32_t rem, uint32_t base, uint32_t exp) {
    if (base & 2) {
        /* divider is considered negative */
        base = -base;
        /* check if result should be negative */
        if (exp & 1) {
            rem = -rem;
        }
    }
    return (mbin_exp_32(rem, mbin_log_32(0, base) * exp));
}
```